Virtual Network For Virtual Machine Communication And Migration

DOCUMENT ID	DATE PUBLISHED
US 10228959 B1	2019-03-12

INVENTOR INFORMATION

NAME	CITY	STATE	ZIP CODE	COUNTRY
Anderson; Evan K.	Seattle	WA	N/A	US
Petrescu-Prahova; Cristian	Seattle	WA	N/A	US
Beda, III; Joseph S.	Seattle	WA	N/A	US

ASSIGNEE INFORMATION

NAME	CITY	STATE	ZIP CODE	COUNTRY
Google LLC	Mountain View	CA	N/A	US

TYPE CODE
02

APPLICATION NO	DATE FILED
13/186293	2011-07-19

DOMESTIC PRIORITY (CONTINUITY DATA)

us-provisional-application US 61492708 20110602

US CLASS CURRENT:

1/1

CPC CURRENT

TYPE	CPC	DATE
CPCI	G 06 F 9/546	2013-01-01
CPCI	G 06 F 9/5077	2013-01-01
CPCI	G 06 F 9/45558	2013-01-01
CPCI	G 06 F 9/4856	2013-01-01

KWIC Hits

APPLICATION FILING DATE

20110719

Abstract

Systems and techniques are described for virtual machine communication and migration. A described technique includes operating server systems that are configured to run virtual machines and providing a virtual network for Internet Protocol (IP) based communications to the virtual machines. The virtual machines can be assigned network addresses, such as IP addresses, on the virtual network. Providing the virtual network can include using separate IP tunnels to effect delivery of IP packets on the virtual network to the virtual machines, respectively.

The technique includes migrating a virtual machine running on a first server system to a second server system. The migrated virtual machine can maintain its assigned IP address. The technique includes updating a tunnel endpoint destination associated with the assigned IP address of the migrated virtual machine. The updated tunnel endpoint destination can be based on a network address associated with the second server system.

Background/Summary

PRIORITY CLAIM (1) This patent document claims the benefit of the priority of U.S. Provisional Application Ser. No. 61/492,708, filed Jun. 2, 2011 and entitled “Virtual Network For Virtual Machine Communication And Migration,” which is incorporated herein by reference in its entirety.

BACKGROUND

(1) This patent document relates to virtual machine communications.

(2) Cloud computing is network-based computing in which typically large collections of servers housed in data centers or “server farms” provides computational resources and data storage as needed to remote end users. Some cloud computing services allow end users to run user-provided server software applications (e.g., e-commerce server application, web server, or file server) in the cloud. Some other cloud computing services provide access to software applications such as word processors and other commonly used applications to end users who interface with the applications through web browsers or other client-side software. Users' electronic data files are usually stored in the server farm rather than on the users' computing devices. Maintaining software applications and user data on the server farm may simplify management of end user computing devices. Some cloud computing services allow end users to execute software applications in virtual machines.

SUMMARY

(3) In general, one aspect of the subject matter described in this document can be embodied in a technique that can include operating server systems that are configured to run virtual machines and providing a virtual network for Internet Protocol (IP) based communications to the virtual machines. The virtual machines can be assigned IP addresses on the virtual network. Providing the virtual network can include using separate IP tunnels to effect delivery of IP packets on the virtual network to the virtual machines, respectively. The technique can include migrating a virtual machine running on the first server system to the second server system. The migrated virtual machine can maintain its assigned IP address. The technique can include updating a tunnel endpoint destination associated with the assigned IP address of the migrated virtual machine. The updated tunnel endpoint destination can be based on a network address associated with the second server system. Other embodiments of this aspect include corresponding systems, apparatus, and computer programs encoded on computer storage devices.

(4) These and other aspects can optionally include one or more of the following features. Providing the virtual network can include using a first network agent to encapsulate an IP packet sent by a first virtual machine of the virtual machines and to send the encapsulated IP packet via an IP tunnel. Providing the virtual network can include using a second network agent to receive the encapsulated IP packet via the IP tunnel and to deliver a version of the encapsulated IP packet to a second virtual machine of the virtual machines. In some cases, the first virtual machine and the second virtual machine run on the first server system. In some cases, the second virtual machine is the migrated virtual machine, where a destination address of the encapsulated IP packet is based on the updated tunnel endpoint destination.

(5) Migrating the virtual machine can include operating the first server system to continue to receive one or more inbound IP packets via the virtual network for the migrated virtual machine and operating the first server system to send the one or more inbound IP packets to the migrated virtual machine using an IP tunnel that is based on the updated tunnel endpoint destination. Migrating the virtual machine can include sending virtual machine state information associated with the migrated virtual machine to the second server system; and operating the second server system to resume operations of the migrated virtual machine using the virtual machine state information.

(6) Implementations can include detecting a reduced processing capacity at the first server system, where migrating the virtual machine is responsive to the detected reduced processing capacity. Implementations can include detecting a hardware failure on the first server system, where migrating the virtual machine is responsive to the detected hardware failure. Implementations can include receiving a migration command, where migrating

the virtual machine is responsive to the migration command. In some implementations, the second server system runs a backup virtual machine that is synchronized to the virtual machine running on the first server system, and where migrating the virtual machine can include activating the backup virtual machine. The first server system and the second server system can reside in different geographic locations.

(7) A system can include a controller and server systems that are configured to run virtual machines that communicate on a virtual network, the virtual machines being assigned IP addresses on the virtual network, where the server systems use separate IP tunnels to effect delivery of IP packets on the virtual network to the virtual machines, respectively, where the server systems include a first server system and a second server system. The controller configured to migrate a selected virtual machine running on the first server system to the second server system, the migrated virtual machine maintaining an assigned IP address, where the controller is configured to update a tunnel endpoint destination associated with the assigned IP address of the migrated virtual machine, the updated tunnel endpoint destination being based on a network address associated with the second server system.

(8) In some implementations, the first server system is configured to use a first network agent to encapsulate an IP packet sent by a first virtual machine of the virtual machines and to send the encapsulated IP packet via an IP tunnel, and the second server system is configured to use a second network agent to receive the encapsulated IP packet via the IP tunnel and to deliver a version of the encapsulated IP packet to a second virtual machine of the virtual machines. In some implementations, the first virtual machine and the second virtual machine run on the first server system. In some implementations, the second virtual machine is the migrated virtual machine, where a destination address of the encapsulated IP packet is based on the updated tunnel endpoint destination.

(9) The first server system can be configured to continue to receive one or more inbound IP packets via the virtual network for the migrated virtual machine, and send the one or more inbound IP packets to the migrated virtual machine using an IP tunnel that is based on the updated tunnel endpoint destination. The second server system can be configured to receive virtual machine state information associated with the selected virtual machine, and resume operations of the selected virtual machine using the virtual machine state information. In some implementations, the second server system runs a backup virtual machine that is synchronized to the selected virtual machine and activates the backup virtual machine as the migrated virtual machine. In some implementations, the first server system includes at least a portion of the controller. The first server system and the second server system can reside in different geographic locations.

(10) The controller can be configured to detect a reduced processing capacity at the first server system, where the selected virtual machine is selected based on the detected reduced processing capacity. In some implementations, the controller is configured to perform a virtual machine migration based on detecting a hardware failure on the first server system. In some implementations, the controller is configured to perform a virtual machine migration based on receiving a migration command.

(11) In another aspect, a system can include server systems that are configured to run virtual machines that communicate on a virtual network, the virtual machines being assigned network addresses on the virtual network, where the server systems use separate tunnels to effect delivery of packets on the virtual network to the virtual machines, respectively, where the server systems include a first server system and a second server system. In some implementations, the server systems are configured to migrate a selected virtual machine running on the first server system to the second server system, the migrated virtual machine maintaining its assigned network address on the virtual network. The server systems can be configured to update a tunnel endpoint destination associated with the assigned network address of the migrated virtual machine, the updated tunnel endpoint destination being based on a network address associated with the second server system.

(12) Particular embodiments of the subject matter described in this document can be implemented so as to realize one or more of the following advantages. A virtual network can provide a virtual communication medium to connect virtual machines that are running on one or more server systems. The virtual network, which can be distributed over geographically separated locations, can provide a virtual machine communication medium that is independent of physical location. Virtual machines on the virtual network can be migrated between different server systems that respectively reside in geographically separated locations. Migrated virtual machines can resume communications on the virtual network with little or no communication disruption (e.g., virtual network traffic can be rerouted to the new server system location of a migrated virtual machine).

(13) The details of one or more embodiments of the subject matter described in this specification are set forth in the accompanying drawings and the description below. Other features, aspects, and advantages of the subject matter will become apparent from the description, the drawings, and the claims.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. 1A shows an example of a distributed system that runs multiple virtual machine instances over a geographically distributed virtual network.

(2) FIG. 1B shows another example of a distributed system that runs multiple virtual machine instances over a geographically distributed virtual network.

(3) FIG. 2 shows an example of token negotiating and message sending.

(4) FIG. 3 shows an example of an encapsulated packet.

(5) FIG. 4 shows an example of a technique for packet de-encapsulation.

(6) FIG. 5 shows an example of a distributed system process that includes virtual machine migration.

(7) FIG. 6 shows an example of a networking architecture for a distributed system configured to run virtual machines.

(8) FIG. 7A shows a before-migration snapshot of a virtual network routing table.

(9) FIG. 7B shows an after-migration snapshot of a virtual network routing table.

(10) FIG. 8 shows an example of a migration process for virtual machines communication via a virtual network.

(11) Like reference numbers and designations in the various drawings indicate like elements.

DETAILED DESCRIPTION

(12) FIG. 1A shows an example of a distributed system that runs multiple virtual machine instances over a geographically distributed virtual network. A distributed system **100** can run virtual machines (VMs) **110**, **112**, **114**, on host machines **102**, **104**. Elements of the distributed system **100** can reside in different physical locations. For example, a first host machine **102** can reside in a first data center, while a second host machine **104** can reside in a second data center. In some implementations, these data centers can be located in different states, countries, or both.

(13) The distributed system **100** can provide a virtual network **140** to the virtual machines **110**, **112**, **114** for Internet Protocol (IP) based communications. The VMs **110**, **112**, **114** are assigned network addresses (e.g., an IPv4 address or IPv6 address) that are routable on the virtual network **140**. In some implementations, the virtual network **140** includes a private subnet (e.g., 192.168.0.0/16, 10.0.0.0/8).

(14) IP traffic on the virtual network **140** is carried by IP tunnels. Host machines **102**, **104** perform IP encapsulation and de-encapsulation **142** for IP traffic to and from each VM **110**, **112**, **114**. Encapsulated IP traffic from one VM is sent to another VM via IP tunneling **144**. An IP tunnel provides a transport layer. For example, IP tunneling can include sending an IP tunnel packet that includes an encapsulated packet. The encapsulated packet can be an IP packet. Other types of encapsulated packets are possible. In some cases, an IP tunnel can both originate and terminate on the same host machine (e.g., source and destination VMs are on the same host machine). In some cases, an IP tunnel can originate at one host machine and terminate at a different host machine (e.g., source and destination VMs are on different host machines). IP tunnel traffic between the host machines **102**, **104** can be carried over a virtual private network (VPN) via a network **150** such as the Internet.

(15) Host machines **102**, **104** can use a virtual network routing table to retrieve the endpoint locations of the IP tunnels. In some implementations, based on an input VM identifier or VM IP address, a virtual network routing

table provides a routable IP address on network **150** that corresponds to the host machine that runs the VM associated with the input VM identifier or VM IP address.

(16) Based on the virtual network **140** spanning multiple host machines **102**, **104** in geographically separate areas, VM migration can occur with minimal or zero disruption to IP traffic from or to a VM migrated that has been migrated between geographically separate areas. For example, a VM **112** can be migrated **160** from a first host machine **102** at a first location to a second host machine **104** at a second location. Endpoints for the tunnels associated with VM **112** can be updated to reflect the change in host machine. In some implementations, the host machines **102**, **104** can run multiple virtual networks. For example, two virtual machines can be on the same physical machine, but attached to different virtual networks. Furthermore, a virtual machine can have one or more virtual network interface cards (NICs) that are attached to one or more virtual networks. Each virtual network interface can be assigned a network address (e.g., IPv4 address, IPv6 address, or a layer 2 Media Access Control address). In some implementations, a virtual machine stores virtual network addresses and their associated network identifiers.

(17) FIG. 1B shows another example of a distributed system that runs multiple virtual machine instances over a geographically distributed virtual network. Host machines **102**, **104** included in a distributed system **101** can contain one or more data processing apparatuses such as rack mounted servers or other computing devices. Host machines **102**, **104** can have different capabilities and computer architectures. Host machines **102**, **104** can communicate with each other through a network such as a private network **116** (e.g., dedicated or leased optical fiber or copper wires). Host machines **102**, **104** can also communicate with devices on external networks, such as the Internet **122**, through one or more gateways **120** which are data processing apparatus responsible for routing data communication traffic between the private network **116** and the Internet **122**. Other types of external networks are possible.

(18) The private network **116** can include one or more wired (e.g., Ethernet) or wireless (e.g., WI-FI) networks, for example. In some implementations, the private network **116** includes physical communication assets such as optical fiber or copper wire that connect two data centers. In some implementations, the private network **116** is implemented over an external network such as the Internet **122**. In some implementations, data centers have respective local area networks (LANs) that are aggregated together to form a private network **116**. IP traffic between data centers can be routed, for example, by using Multiprotocol Label Switching (MPLS). Other types of routing are possible.

(19) Each host machine **102**, **104** executes a host operating system **106**, **108**. A host operating system **106**, **108** manages host machine resources. In this example, host operating systems **106**, **108** run software that virtualizes the underlying host machine hardware and manages concurrent execution of one or more VMs. For example, the host operating system **106** manages two VMs (VM **110** and VM **112**), while a different host operating system **108** manages a single VM **114**. A host machine can, in general, manage larger quantities of virtual machines; however, the quantity may be limited based on physical resources of the host machine.

(20) The VMs **110**, **112**, **114** use a simulated version of an underlying host machine hardware, which can be referred to as virtual hardware (e.g., virtual hardware **110***a*, **112***a* and **114***a*). Software that is executed by the virtual hardware can be referred to as guest software. In some implementations, guest software cannot determine if it is being executed by virtual hardware or by a physical host machine. If guest software executing in a VM, or the VM itself, malfunctions or aborts, other VMs executing on the host machine will not be affected. A host machine's microprocessor(s) can include processor-level mechanisms to enable virtual hardware to execute software applications efficiently by allowing guest software instructions to be executed directly on the host machine's microprocessor without requiring code-rewriting, recompilation, or instruction emulation.

(21) Each VM (e.g., VMs **110**, **112** and **114**) is allocated a set of virtual memory pages from the virtual memory of the underlying host operating system and is allocated virtual disk blocks from one or more virtual disk drives for use by the guest software executing on the VM. For example, host operating **106** allocates memory pages and disk blocks to VM **110** and VM **112**, and host operating system **108** does the same for VM **114**. In some implementations, a given VM cannot access the virtual memory pages assigned to other VMs. For example, VM **110** cannot access memory pages that have been assigned to VM **112**. A virtual disk drive can be persistent across VM restarts. In some implementations, virtual disk blocks are allocated on physical disk drives coupled to host machines. VMs can be allocated network addresses through which their respective processes can communicate with other processes via a virtual network. For example, guest software executing on VM **110** can

communicate with guest software executing on VM **112** or VM **114** via the virtual network using the allocated network addresses.

(22) A VM's guest software can include a guest operating system (e.g., guest operating systems **110***b*, **112***b* and **114***b*) which is software that controls the execution of respective guest software applications (e.g., guest applications **110***c*, **112***c* and **114***c*), within the VM and provides services to those applications. For example, a guest operating system can be an operating system such as a variant of the UNIX operating system. Other operating systems are possible. Each VM can execute the same guest operating system or different guest operating systems. Booting a VM can include using a virtual boot disk to load a kernel image associated with a guest operating system into memory. A guest operating system's access to resources such as networks and virtual disk storage is controlled by the underlying host operating system.

(23) By way of illustration, and with reference to virtual machine **110**, when the guest application **110***c* or guest operating system **110***b* attempts to perform an input/output operation on a virtual disk, initiate network communication, or perform a privileged operation, for example, the virtual hardware **110***a* is interrupted so that the host operating system **106** can perform the action on behalf of the virtual machine **110**. The host operating system **106** can perform these actions with a process that executes in kernel process space **106***b*, user process space **106***a*, or both.

(24) The kernel process space **106***b* is virtual memory reserved for the host operating system **106**'s kernel **106***d* which can include kernel extensions and device drivers, for instance. The kernel process space has elevated privileges (sometimes referred to as “supervisor mode”); that is, the kernel **106***d* can perform certain privileged operations that are off limits to processes running in the user process space **106***a*. Examples of privileged operations include access to different address spaces, access to special functional processor units in the host machine such as memory management units, and so on. The user process space **106***a* is a separate portion of virtual memory reserved for user mode processes. User mode processes cannot perform privileged operations directly.

(25) In various implementations, a portion of VM network communication functionality is implemented in a communication process (e.g., communication process **106***c*). In some implementations, the communication process executes in the user process space (e.g., user process space **106***a*) of a host operating system (e.g., host operating system **106**). In other implementations, the communication process can execute in the kernel process space (e.g., kernel process space **106***d*) of the host operating system. There can be a single communication process for all VMs executing on a host machine or multiple communication processes, one for each VM executing on a host machine. In yet further implementations, some portion of the communication process executes in the user process space and another portion executes in the kernel process space. In some implementations, the communication process can use a hardware level encapsulation mechanism.

(26) A communication process **106***c*, **108***c* can communicate with a VM registry service **118** to establish a virtual network pair (VNP) between two VMs. In a virtual network, a VNP can be used to route traffic between two endpoints using one or more virtual connections or links. Network traffic associated with the VNP can be carried via a local loop (e.g., if both VMs are on the same host machine) or carried via one or more networks, such as a private network **116**, Internet **122**, or a combination thereof. For example, a VNP between virtual machine **110** and virtual machine **114** can be used to route packets sent between VNP endpoints managed respectively by first and second communication processes **106***c*, **108***c* over the private network **116**. In some implementations, a server such as a VM registry server implements the functionality of the VM registry service. The VM registry service **118** can manage assignments of network addresses (e.g., IP addresses) to VMs, and maintain mappings between VM network addresses on a virtual network and the respective network addresses of the host machines running the VMs.

(27) FIG. 2 shows an example of token negotiating and message sending. Before a VM can send packets to or receive packets from other VMs, a network agent for a VM can obtain a secret key that corresponds to the VM. In some implementations, a network agent is included in a communication process executing on the VM's host machine.

(28) A secret key is a piece of information that serves to uniquely identify a VM among all VMs associated with a virtual network (e.g., virtual network **140**). The secret key can be a number, a string of characters, other data, combinations of these, or any other string of bits of sufficient entropy. The secret key can be generated in such a way that an entity with no prior knowledge of the key has no computationally feasible way of deriving the key.

The secret key can be generated using, for example, a cryptographically secure pseudorandom number generator. In some implementations, the secret key is assigned to a VM by a VM registry service. Secret keys can also be assigned by another process besides the VM registry service. A given VM may not know what its secret key is and does not know the secret key of other VMs. In some implementations, a communication process (e.g., communication process **106***c*) on a given host machine (e.g., host machine **102**) keeps track of the secret keys for VMs managed by the host operating system (e.g. host operating system **106**).

(29) The communication process **204** (COM A process **204**) on VM A **202**'s host machine can request a secret key for VM A **202** from the VM registry service **206** by sending a message **212** to the VM registry service **206**. The request for the secret key can be sent via secure protocol that allows the VM registry service **206** to authenticate which communication process it is communicating with. Other communications between the VM registry service and COM A process **204** or other communication processes can also use the same secure protocol.

(30) The VM registry service **206** responds to the COM A process **204** with a message **214** containing the secret key for VM A. As a further example, the communication process **208** (COM B process **208**) on VM B **210**'s host machine can request a secret key for VM B **210** from the VM registry service **206** by sending a message **216** to the VM registry service **206**. The VM registry service **206** responds to the COM B process **208** with a message **218** containing the secret key for VM B.

(31) Before an initial packet from one VM to another is transmitted, a VNP between the two VMs is established. In various implementations, the communication process on a given host machine is responsible for establishing VNPs. For example, communication process **106***c* is responsible for establishing VNPs for VM **110** and VM **112**. Likewise, communication process **108***c* can do the same for VM **114**. Each VM can communicate with one or more other VMs using a separate VNP for each. Referring again to FIG. 2 and by way of example, VM A **202** attempts to transmit a packet **220** to VM B **210**. In some implementations, the packet is an IP version 4 or version 6 packet. In other implementations, the packet is an Open Systems Interconnection Reference Model layer 3 or higher protocol packet, such as, for example, UDP, Transmission Control Protocol (TCP), and so on. Other types of packets are possible including, for example, Ethernet packets. The packet is intercepted by the COM A process **204** of the host operating system on which VM A **202** is executing. The COM A process **204** determines that a VNP between VM A **202** and VM B **210** has not yet been established. This being the case, the COM A process **204** requests a token to be used to communicate with VM B **210** from the VM registry service **206**. A token is required in order to establish a unidirectional VNP from a source VM to a destination VM. The token request **222** can contain the secret key for VM A **202** and a network address of the destination VM B **210**, for example.

(32) In response to the request **222**, the VM registry service **206** uses the secret key S_a of VM A **202** to look up or determine the following attributes of VM A **202**:

(33) IP_VM_a, the IP address assigned to VM A;

(34) Phys_Port_a, the UDP port assigned to VM A on VM A's host machine;

(35) Phys_IP_a, the IP address of VM A's host machine; and

(36) expiry_a_b, the validity period of the token which, in some implementations, is the current time plus a time-to-live (TTL). The TTL can be on the order of minutes (e.g., 10 minutes) or other granularities. In some implementations, expiry_a_b is set to a value (e.g., −1) to indicate that the token never expires. In some implementations, the concatenation of Phys_Port_a and Phys_IP_a can be considered as a VM host address.

(37) In some implementations, the VM registry service **206** verifies that the request **222** was actually transmitted from Phys_IP_a and otherwise denies the request. In further implementations, the VM registry service **206** can consult a traffic policy to determine if VM A **202** should be allowed to communicated with VM B **210** and, if not, denies the request.

(38) In various implementations, the VM registry service **206** computes the token T_a_b for traffic from VM A **202** to VM B **210** as follows (FIG. 2 at **224**):

$$T_a_b = \text{TruncMAC}(S_b, \text{Phys_IP}_a|\text{Phys_IP}_b|\text{Phys_Port}_a|\text{Phys_Port}_b|\text{IP_VM}_a|\text{IP_VM}_b|\text{expiry}_a_b)$$

where '|' denotes concatenation, S b is VM B **210**'s secret key, and TruncMAC is a Message Authentication Code (MAC) function (e.g., a function based on HMAC-SHA1) that has been truncated, for example, to 64 bits. Other MAC functions are possible. In some implementations, all of the fields being concatenated are fixed size, i.e., the concatenation is uniquely determined without the need for separators. Although the fields are concatenated in a certain order, other field orderings are possible. In some implementations, the concatenation of Phys_Port_b and Phys_IP_b can be considered as a VM host address.

(39) In some implementations, a secret key ties a host machine addressing scheme (e.g., <Phys_IP, Phys_Port>) to a VNP addressing scheme (e.g., IP address of VM on a virtual network). In some implementations, a secret key ties a host machine addressing scheme (e.g., <Phys_IP, Phys_Port>) to a memory region associated with a VM via remote direct memory access (RDMA).

(40) The VM registry service **206** returns the tuple (Phys_IP_b, Phys_Port_b, T_a_b, expiry_a_b) to COM A process **204** in a response **226** to the request **222**, where Phys_IP_b is the IP address of the host machine on which VM B **210** is executing and Phys_Port_b is the UDP port on VM B's host machine that has been reserved for receiving traffic directed to VM B **210**. Once the COM A process **204** receives the tuple, the packet **220** can be encapsulated, at **228**.

(41) At **230**, the encapsulated packet is transmitted to VM B's host machine using the Phys_IP_b and Phys_Port_b as the destination address of the packet. Future packets destined for VM B **210** can reuse the tuple information until expiry_a_b has passed. Once the expiry_a_b has passed (if it is not set to a value indicating that the token never expires), a new token can be obtained as described above, for example. For VM B **210** to send packet to VM A **202**, a token can be used to establish a unidirectional VNP from VM B **210** to VM A **202**. The same process can be followed as outlined above or, alternatively, a packet sent from VM A to VM B can include the token.

(42) The encapsulated packet is received by the destination host machine for VM B **210** where it is intercepted by the COM B process **208**. At **232**, the COM B process **208** de-encapsulates the packet and verifies it based on VM B's secret key. At **234**, the communication process **208** the COM B process sends the de-encapsulated packet to VM B **210**. This can be done by injecting the IP packet into the network stack as an Ethernet frame, for example.

(43) FIG. 3 shows an example of an encapsulated packet. A communication process can receive an outgoing packet generated by a VM. The outgoing packet is encapsulated in a data portion **302***e* of a UDP packet **302**. In other words, the data portion **302***e* includes an encapsulated packet, such as an encapsulated IP packet **304***c*, that includes the outgoing packet.

(44) The UDP packet **302** contains a header that includes a source port number **302***a* (16 bits), a destination port number **302***b* (16 bits), a length field **302***c* (16 bits) which is the length of the data portion **302***e*, and a checksum **302***d* (16 bits). In various implementations, the source port **302***a* is set to Phys_Port_a and the destination port **302***b* is set to Phys_Port_b. The data portion **302***e* is variable length and contains the outgoing packet. In some implementations, the data portion **302***e* can be encrypted. Symmetric or asymmetric encryption key algorithms can be used to encrypt some or all of the data portion **302***e*, for example. The encryption keys can be distributed by a VM registry service, e.g., VM registry service **118** of FIG. 1B. In some implementations, a conventional key negotiation protocol, e.g., the Diffie-Hellman protocol, can be used in an encryption of the data portion **302***e.*

(45) The UDP packet's data portion **302***e* contains a VNP packet **304**. A VNP packet **304** can include the token Tab **304***a* (64 bits), the expiry time expiry_a_b **304***b* (32 bits), the outgoing packet **304***c* (variable length), and an optional options field **304***d* (variable length). Other lengths for the VNP packet fields are possible. In addition, the fields of the VNP packet **304** can occur in a different order than that which is illustrated in FIG. 3. In some implementations, the outgoing packet is an IP packet. Other outgoing packet types are possible including, for example, Ethernet packets. Once encapsulation is complete, the communication process or another process on the respective host machine's operating system can transmit the UDP packet **302** in an IP packet with a source IP address of Phys_IP_a and a destination IP address of Phys_IP_b. Alternatively, the encapsulated packet can be transmitted using a different protocol.

(46) In some implementations, the VNP packet **304** includes an options field **304***d*. The options field **304***d* allows a sending VM to convey additional information to a receiving VM. If the encapsulated packet is a UDP packet, as illustrated in FIG. 3, the length of the options field can be calculated based on the difference of the value

specified in the length **302***c* field and the combined size of the token **304***a*, expiry **304***b*, and encapsulated IP packet **304***c* fields. The size of the encapsulated IP packet **304***c* is available in the header of the packet **304***c*. The options field **304***d* can be omitted from the VNP packet **304** if its inclusion would push the size of the encapsulated packet (e.g., UDP packet **302**) above the maximum transmission unit (MTU) which would result in packet fragmentation. In further implementations a field such as a bit flag, for example, can be added to the VNP packet **304** to indicate the presence of an options field **304***d.*

(47) The options field **304***d* can be used to send a return VNP token T_b_a to the receiving VM in order to avoid the receiving VM having to obtain a token in order to reply to the sending VM. The COM A process **204** for VM A **202** has all information on hand that is needed to compute T_b_a; in particular, the token is signed with VM A **202**'s secret key S_a. The return token T_b_a can likely be included in the first packet sent between VM A **202** and VM B **210** since, in the case of TCP, the first packet is a small SYN packet which leaves sufficient room for the options field **304***d* containing the return token without the resulting encapsulated packet exceeding MTU. The options field **504***d* can also be used to send various protocol extensions, to exchange encryption keys, to send control flow information, and so on.

(48) FIG. 4 shows an example of a technique **400** for packet de-encapsulation as can be performed by the communication process **208** or another process on the destination host machine. The de-encapsulation process can be performed without the need to communicate with the VM registry service **118**. With reference to FIG. 2, when an encapsulated packet is received fields from the packet are extracted in order to validate the packet (step **402**). The source and destination IP addresses and ports of the UDP packet are extracted: Phys_IP_a, Phys_IP_b, Phys_Port_a, and Phys_Port_b. The source and destination address of the encapsulated IP datagram are also extracted: IP_VM_a, IP_VM_b. Finally, the token and expiry are extracted from the encapsulation header: T_a_b and expiry_a_b.

(49) If expiry_a_b is less than the current time minus a skew (step **404**), the encapsulated packet is discarded (step **416**). In some implementations, the skew is an estimate of likely clock skew between host machine clocks (e.g., a few seconds). The skew can be set to zero if host machine clocks are synchronized. Otherwise, if expiry_a_b is greater than the current time plus skew and TTL (step **406**), the encapsulated packet is also discarded (step **416**). A check is performed to verify whether the host operating system is hosting a VM having the address VM_IP_b (step **408**). If not, the encapsulated packet is discarded (step **416**). The extracted fields and VM B **210**'s secret key S b are used to calculate T_a_b_actual as follows (step **410**):

*T_a_b*_actual=TruncMAC(*S_b*,Phys_IP_*a*|Phys_IP_*b*|Phys_Port_*a*|Phys_Port_*b*|IP_VM_*a*|IP_VM_*b*|expiry_*a*_*b*)
.

(50) T_a_b_actual is then compared to the token Tab from the encapsulated packet (step **412**). If T_a_b_actual is the same as T_a_b then the encapsulated packet (e.g., encapsulated IP packet **304***c*) is extracted from the encapsulated packet (e.g., encapsulated packet **302**) and is delivered to VM B **210** (step **414**). Otherwise, the encapsulated packet is discarded (step **416**).

(51) A gateway (e.g., gateway **120**) can serve as an encapsulation gateway to allow virtual machines on a virtual network to communicate with endpoints on an external network such as the Internet. A host machine can send encapsulated packets to the gateway. When the gateway receives encapsulated packet from a VM destined for the external network, the gateway can validate the token of the VNP packet. If the token validation fails, the packet is discarded. If validation succeeds, the encapsulated packet is de-encapsulated and the encapsulated packet is extracted and injected into the gateway's network stack where it is subject to normal routing.

(52) Upon receiving, from the external network, a non-encapsulated packet destined for a VM, the gateway obtains a token for the destination of the packet and then encapsulates the packet. In some implementations, obtaining a token can include communicating with a VM registry service. In some implementations, obtaining a token can include retrieving a token from a local cache of tokens. The gateway **120** then transmits the encapsulated packet to the destination VM's host machine.

(53) If a communication process associated with a VM queries a VM registry service for a token that is for a destination computer that is not on the virtual network, the VM registry service responds with a tuple (Phys_IP_b, Phys_Port_b, T_a_b, expiry_a_b) where Phys_IP_b, Phys_Port_b and Tab are for the gateway, rather than the ultimate destination, so that the VM's packets are routed through, and de-encapsulated by, the gateway. In some

implementations, the VM can use this tuple to send packets to any destination on an external network thus avoiding having to obtain a tuple from the VM registry service for each unique destination.

(54) FIG. 5 shows an example of a distributed system process that includes virtual machine migration. A distributed system can include multiple server systems. At **505**, a distributed system process includes operating server systems that are configured to run virtual machines. Such server systems can reside in different geographic locations. A server system can include one or more host machines. Operating server systems can include sending control messages to boot virtual machines on one or more machines. Operating server systems can include running a scheduling routine configured to distribute a compute load between one or more virtual machines.

(55) At **510**, the distributed system process includes providing a virtual network for IP based communications to the virtual machines, the virtual machines being assigned IP addresses on the virtual network. Providing the virtual network can include using separate IP tunnels to effect delivery of IP packets to the virtual machines, respectively. Providing the virtual network can include using a first network agent to encapsulate an IP packet sent by a first virtual machine and to send the encapsulated IP packet via an IP tunnel. Providing the virtual network can include using a second network agent to receive the encapsulated IP packet via the IP tunnel and to deliver a version of the encapsulated IP packet to a second virtual machine.

(56) At **515**, the distributed system process includes migrating a virtual machine running on a first server system to a second server system, the migrated virtual machine maintaining its assigned IP address. Migrating the virtual machine can include sending virtual machine state information associated with the migrated virtual machine to the second server system. Migrating the virtual machine can include operating the second server system to resume operations of the migrated virtual machine using the virtual machine state information. The distributed system process can include selecting a virtual machine to migrate based on a detection of an event such as a hardware failure event, a processing load event, or a migration command that identifies one or more virtual machines for migration.

(57) At **520**, the distributed system process includes updating a tunnel endpoint destination associated with the assigned IP address of the migrated virtual machine. The updated tunnel endpoint destination can be based on a network address associated with the second server system. In some implementations, updating a tunnel endpoint destination includes sending information to a VM registry service, the information indicating the new home (e.g., new server system) of the migrated virtual machine.

(58) In some implementations, migrating a virtual machine can include forwarding network traffic for a duration of time. For example, migrating the virtual machine can include operating the first server system to continue to receive one or more inbound IP packets via the virtual network for the migrated virtual machine and operating the first server system to send the one or more inbound IP packets to the migrated virtual machine using an IP tunnel that is based on the updated tunnel endpoint destination.

(59) A distributed system can provide a high-availability capability. Pairs of virtual machines can be run on separate host machines, where one of the virtual machines mirrors the other. In some implementations, a system runs primary and secondary VMs in lock step with mirrored state. If one of the virtual machine fails unexpectedly, the other virtual machine takes over. In some implementations, a host machine process that controls a primary VM of a VM pair sends periodic state information updates of the primary VM to a host machine process that controls a secondary VM of the VM pair. The primary and secondary host machine processes can exchange heartbeat messages, and based on a lack of receiving a heartbeat message, the secondary host machine processes can cause primary VM traffic on a virtual network to be rerouted to the secondary VM and cause the secondary VM to become active. Migrating the virtual machine can include activating a secondary VM based on a lack of receiving a heartbeat message. In some implementations, migrating a virtual machine includes migrating the virtual machine to a new physical endpoint on the same host machine.

(60) FIG. 6 shows an example of a networking architecture for a distributed system configured to run virtual machines. A distributed system can contain server systems that include host machines **615***a*, **615***b*, which are configured to run one or more virtual machines **620***a*, **620***b*, **620***c*, **620***d*. Host machines **615***a*, **615***b* use IP tunnels **660***a*, **660***b*, **660***c* to carry IP traffic between virtual machines **620***a-d* and IP traffic between the virtual machines **620***a-d* and VM gateways **640***a*, **640***b*. The VM gateways **640***a, b* are configured to provide Internet **650** access to the virtual machines **620***a-d.*

(61) The host machines **615***a*, **615***b* can connect to the Internet **650** via respective local area networks (LANs) **655***a*, **655***b*. In some implementations, host machines **615***a*, **615***b* are assigned IP addresses (e.g., **134**.**130**.**78**.**11**, **224**.**10**.**202**.**6**) that are visible on the Internet **650**. Traffic on an inter-host-machine IP tunnel **660***a* (e.g., IP tunnel packets) can be routed from the host machine **615** on a first LAN **655***a* to a host machine **615***b* on a second LAN **655***b* via the Internet **650** (tunnel **660***a* carries traffic between VM-**1 620***a* and VM-**13 620***c*). Traffic on an intra-host-machine IP tunnel **660***b* can be routed completely inside a host machine **615***a.*

(62) The VM gateway **640***a*, **640***b* can route traffic between the Internet **650** and the virtual network **601** by changing source or destination address information in packet headers so that the address space of the virtual network **601** is hidden behind the IP address of the gateway **640***a*,**640***b*. In particular, packets arriving at the gateway **640***a*,**640***b* from the Internet **650** and bound for the virtual network **601** have their IP datagram destination address changed from that of the gateway **640***a*, **640***b* (e.g., 134.130.78.1, or 224.10.202.1) to the IP address of a destination VM (e.g., 192.168.1.10). Packets arriving at the gateway **640***a*, **640***b* from the virtual network **601** and bound for the Internet **650** have their IP source address changed from that of a VM (e.g., 192.168.1.10) to the IP address of the VM gateway **640***a*, **640***b* visible on the Internet **650** (e.g., **134**.**130**.**78**.**1**, **224**.**10**.**202**.**1**). Based on the virtual network **601** using a private subnet, the VM gateways **640***a, b* can perform network address translation (NAT) to translate between private subnet addresses and public addresses.

(63) Virtual machines **620***a*, **620***b*, **620***c*, **620***d* running on the host machines **615***a-b* can run processes such as webserver processes that interact with browser software running on clients **630***a*, **630***b*, **630***c*. A client can be a data processing apparatus such as, for example, a server, a personal computer, a laptop computer, a smart phone, an embedded device (e.g., a smart power meter), or a tablet computer. In this example, client **630***c* communicates with a VM (e.g., VM-**21 620***d*) by sending IP packets to a VM gateway **640***b*. In turn, the VM gateway **640***b* encapsulates the packets from the client **630***c* and sends the encapsulated packets to the host machine **615***b* running VM-**21 620***d* via an IP tunnel **660***c*. The host machine **615***b* running VM-**21 620***d* can de-encapsulate the packets and deliver the packets to VM-**21 620***d.*

(64) FIGS. 7A and 7B show different snapshots of an example virtual network routing table. Machines such as host machines **615***a*, **615***b* and VM gateways **640***a*, **640***b* can use a virtual network routing table to determine where to send a packet having a destination on the virtual network **601**. Tunnel endpoint destinations can be determined based on the virtual network routing table. In this example, a virtual machine **620***b* with identifier VM-**7** is being migrated from a first host machine **615***a* to a second host machine **615***b*. In some implementations, a controller (not shown) can be used to migrate a selected virtual machine from the first host machine **615***a* to the second host machine **615***b*. A controller can be a server system that includes one or more processors. In some implementations, a host machine includes a controller.

(65) FIG. 7A shows a before-migration snapshot of a virtual network routing table. The virtual network routing table **710** includes routing entries for respective virtual machines. For example, the virtual network routing table **710** includes an entry for VM-**7** that directs virtual network traffic to VM-**7** (e.g., IP packets with a destination IP address that corresponds to the IP address assigned to VM-**7**) to the first host machine **615***a*. The entry for VM-**7** includes an Internet IP address assigned to the first host machine **615***a* and a port value that is assigned to VM-**7**.

(66) FIG. 7B shows an after-migration snapshot of a virtual network routing table. Based on the migration of VM-**7** from the first host machine **615***a* to the second host machine **615***b*, the updated virtual network routing table **720** shows that the new entry for VM-**7** includes the Internet IP address assigned to the second host machine **615***b* and a port value that is assigned to VM-**7**. Note that a port value assigned to a VM can change during a migration, as port value assignments can be controlled by the host machine.

(67) FIG. 8 shows an example of a migration process for virtual machines communication via a virtual network. A distributed system can run a migration process. A migration process can be a distributed process running on one or more processors, servers, or a combination thereof. At **805**, a migration process detects a migration trigger event at a first server system. Various examples of a migration trigger event include a planned outage, hardware failure (e.g., a failed or failing hard drive), resource utilization overcapacity (e.g., processor utilization overcapacity, network utilization overcapacity), or an explicit migration command. For example, an administrator may need to shut down a server system to perform maintenance (e.g., replace a piece of hardware in the server system or install software updates to a host operating system) or system upgrades. The administrator can issue one or more commands to a host machine to migrate one or more virtual machines that are running on the host machine. In some implementations, virtual machines running on a host machine can be migrated to different host

machines to distribute the processing load. In some implementations, detecting a migration trigger event includes detecting a reduced processing capacity at a server system.

(68) At **810**, the migration process identifies a virtual machine to migrate. In some cases, the identified machine is indicated by the migration trigger event. For example, a migration trigger message can indicate that a physical machine is failing (e.g., experiencing an increased number of hard-drive errors, or an increased operating temperature). Identifies a virtual machine to migrate can include determining all of the virtual machines that are running on a failing physical machine. For load balancing, identifying a virtual machine to migrate can include selecting a virtual machine based on real-time performance monitoring. Performance monitoring can include monitoring disk usage, network usage, and CPU usage. Other types of monitoring are possible. At **815**, the migration process suspends the virtual machine. Suspending the virtual machine can include writing the state information of the virtual machine (e.g., memory state, CPU state, register state, network packet buffers, etc.) to one or more files. In some cases, a suspend signal can be communicated to the virtual machine to allow the virtual machine to complete one or more operations before it is suspended.

(69) At **820**, the migration process transfers state information of the virtual machine to a second server system. Transferring state information can include sending one or more files to the second server system via a network connection. At **825**, the migration process resumes the virtual machine at the second server system based on the state information. The second server system can initialize a local virtual machine process (e.g., initialize memory, CPU states, register states, network packet buffers, etc.) with the state information. The resumed virtual machine can continue with network communications using the same IP address that it was using when running at the first server system. In an example, the resumed virtual machine, at the second server system, can process and respond to a network packet received when the virtual machine was running at the first server system.

(70) At **830**, the migration process updates a virtual network routing table based on a network address of the second server system. Based on the update, traffic for the virtual machine is sent to the second server system. Updating a virtual network routing table can include sending an update message to one or more gateways.

(71) In some implementations, the migration process includes forwarding virtual network traffic for the virtual machine received at the first server system to the second server system. In some implementations, the second server system runs a backup virtual machine that is synchronized to the virtual machine running on the first server system. Migrating the virtual machine can include activating the backup virtual machine.

(72) Embodiments of the subject matter and the operations described in this specification can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this specification and their structural equivalents, or in combinations of one or more of them. Embodiments of the subject matter described in this specification can be implemented as one or more computer programs, i.e., one or more modules of computer program instructions, encoded on computer storage medium for execution by, or to control the operation of, data processing apparatus. Alternatively or in addition, the program instructions can be encoded on an artificially-generated propagated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus for execution by a data processing apparatus. A computer storage medium can be, or be included in, a computer-readable storage device, a computer-readable storage substrate, a random or serial access memory array or device, or a combination of one or more of them. Moreover, while a computer storage medium is not a propagated signal, a computer storage medium can be a source or destination of computer program instructions encoded in an artificially-generated propagated signal. The computer storage medium can also be, or be included in, one or more separate physical components or media (e.g., multiple CDs, disks, or other storage devices).

(73) The operations described in this specification can be implemented as operations performed by a data processing apparatus on data stored on one or more computer-readable storage devices or received from other sources.

(74) The term “data processing apparatus” encompasses all kinds of apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, a system on a chip, or multiple ones, or combinations, of the foregoing The apparatus can include special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application-specific integrated circuit). The apparatus can also include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an

operating system, a cross-platform runtime environment, a virtual machine, or a combination of one or more of them. The apparatus and execution environment can realize various different computing model infrastructures, such as web services, distributed computing and grid computing infrastructures.

(75) A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, declarative or procedural languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, object, or other unit suitable for use in a computing environment. A computer program may, but need not, correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, sub-programs, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

(76) The processes and logic flows described in this specification can be performed by one or more programmable processors executing one or more computer programs to perform actions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application-specific integrated circuit).

(77) Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read-only memory or a random access memory or both. The essential elements of a computer are a processor for performing actions in accordance with instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto-optical disks, or optical disks. However, a computer need not have such devices. Moreover, a computer can be embedded in another device, e.g., a mobile telephone, a personal digital assistant (PDA), a mobile audio or video player, a game console, a Global Positioning System (GPS) receiver, or a portable storage device (e.g., a universal serial bus (USB) flash drive), to name just a few. Devices suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto-optical disks; and CD-ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(78) To provide for interaction with a user, embodiments of the subject matter described in this specification can be implemented on a computer having a display device, e.g., a CRT (cathode ray tube) or LCD (liquid crystal display) monitor, for displaying information to the user and a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide for interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback; and input from the user can be received in any form, including acoustic, speech, or tactile input. In addition, a computer can interact with a user by sending documents to and receiving documents from a device that is used by the user; for example, by sending web pages to a web browser on a user's client device in response to requests received from the web browser.

(79) Embodiments of the subject matter described in this specification can be implemented in a computing system that includes a back-end component, e.g., as a data server, or that includes a middleware component, e.g., an application server, or that includes a front-end component, e.g., a client computer having a graphical user interface or a Web browser through which a user can interact with an implementation of the subject matter described in this specification, or any combination of one or more such back-end, middleware, or front-end components. The components of the system can be interconnected by any form or medium of digital data communication, e.g., a communication network. Examples of communication networks include a local area network (“LAN”) and a wide area network (“WAN”), an inter-network (e.g., the Internet), and peer-to-peer networks (e.g., ad hoc peer-to-peer networks).

(80) The computing system can include clients and servers. A client and server are generally remote from each other and typically interact through a communication network. The relationship of client and server arises by

virtue of computer programs running on the respective computers and having a client-server relationship to each other. In some embodiments, a server transmits data (e.g., an HTML page) to a client device (e.g., for purposes of displaying data to and receiving user input from a user interacting with the client device). Data generated at the client device (e.g., a result of the user interaction) can be received from the client device at the server.

(81) While this specification contains many specific implementation details, these should not be construed as limitations on the scope of any inventions or of what may be claimed, but rather as descriptions of features specific to particular embodiments of particular inventions. Certain features that are described in this specification in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

(82) Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. In certain circumstances, multitasking and parallel processing may be advantageous. Moreover, the separation of various system components in the embodiments described above should not be understood as requiring such separation in all embodiments, and it should be understood that the described program components and systems can generally be integrated together in a single software product or packaged into multiple software products.

(83) Thus, particular embodiments of the subject matter have been described. Other embodiments are within the scope of the following claims. In some cases, the actions recited in the claims can be performed in a different order and still achieve desirable results. In addition, the processes depicted in the accompanying figures do not necessarily require the particular order shown, or sequential order, to achieve desirable results. In certain implementations, multitasking and parallel processing may be advantageous.

Claims

1. A computer-implemented method performed by one or more computers, the method comprising: operating a plurality of host machines including a first host machine and a second host machine, wherein each host machine is configured to run one or more virtual machines, wherein the first host machine is assigned a first host network address on a first local area network and the second host machine is assigned a second host network address on a different, second local area network, and wherein the one or more virtual machines are configured to communicate over a first virtual network; for each of the one or more virtual machines, assigning only a single respective port of the host machine on which the virtual machine is running to the virtual machine; providing a first virtual network routing table for the first virtual network, wherein the first virtual network routing table includes, for each virtual machine in the first virtual network, only a single respective entry that specifies (i) a virtual network IP address on the first virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) the only single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; migrating a first virtual machine on the first virtual network from the first host machine to the second host machine; before allowing communication on the first virtual network to the first virtual machine after migrating the first virtual machine, updating the only single respective entry in the first virtual network routing table for the first virtual machine such that the host network address, specified in the only single respective entry, is changed from the first host network address to the second host network address while the virtual network IP address on the first virtual network that is assigned to the first virtual machine, specified in the only single respective entry, is maintained as a same virtual network IP address after migrating the first virtual machine on the first virtual network from the first host machine to the second host machine; identifying a message to be routed on the first virtual network, the message received on the first local area network or the second local area network and including a particular host network address and a particular port; verifying that the host machine assigned the particular host network address is running a particular virtual machine assigned the particular port based on a particular entry in the first virtual network routing table including the particular host network address and the particular port; and determining, from the particular entry, a particular virtual network IP address associated with the particular virtual machine on the first virtual network to which to route the message.

2. The method of claim 1, further comprising: providing a second virtual network routing table for a second virtual network, wherein the second virtual network routing table includes, for each virtual machine in the second virtual network, only a single respective entry that specifies: (i) a virtual network IP address on the second virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) only a single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; and operating the first host machine to run the first virtual machine on the first virtual network while running a second virtual machine on the second virtual network.

3. The method of claim 2, wherein the first endpoint of the IP tunnel is the first virtual machine running on the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the second host machine.

4. The method of claim 2, wherein the first endpoint of the IP tunnel is the first virtual machine running on the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the first host machine.

5. The method of claim 1, further comprising: defining, using the first virtual network routing table, an IP tunnel for carrying IP traffic between the first virtual machine on the first virtual network and a second virtual machine on the first virtual network, including defining a first endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the first virtual machine and defining a second endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the second virtual machine.

6. The method of claim 1, wherein the first local area network and the second local area network are connected by a private network.

7. The method of claim 1, wherein the first local area network and the second local area network are connected by the Internet.

8. The method of claim 1, wherein the message includes a validity period of a token, and the token is calculated based on at least a secret key and the validity period.

9. The method of claim 8, further comprising: calculating an actual token based on at least the secret key and the validity period; and comparing the token in the message to the calculated actual token.

10. The method of claim 9, further comprising: in response to the comparison that the calculated actual token is the same as the token in the message, extracting an encapsulated packet and delivering the encapsulated packet to the particular virtual machine.

11. The method of claim 1, wherein updating the only single respective entry in the first virtual network routing table for the first virtual machine such that the host network address, specified in the only single respective entry, is changed from the first host network address to the second host network address while the virtual network IP address on the first virtual network that is assigned to the first virtual machine, specified in the only single respective entry, is maintained as a same virtual network IP address after migrating the first virtual machine on the first virtual network from the first host machine to the second host machine comprises: maintaining, in the single respective entry, the virtual network IP address on the first virtual network that is assigned to the first virtual machine; updating, in the single respective entry, the first host network address that is assigned to the first host machine from which the first virtual machine was migrated from to the second host network address that is assigned to the second host machine to which the first virtual machine is migrated to; and updating, in the single respective entry, the only single respective port of the first host machine, from which the first virtual machine was migrated from, that was assigned to the first virtual machine to the only single respective port of the second host machine, to which the first virtual machine is migrated to, that is assigned to the first virtual machine.

12. A computer program product, encoded on one or more non-transitory computer storage media, comprising instructions that when executed by one or more computers cause the one or more computers to perform operations comprising: operating a plurality of host machines including a first host machine and a second host machine, wherein each host machine is configured to run one or more virtual machines, wherein the first host machine is assigned a first host network address on a first local area network and the second host machine is assigned a second host network address on a different, second local area network, and wherein the one or more

virtual machines are configured to communicate over a first virtual network; for each of the one or more virtual machines, assigning only a single respective port of the host machine on which the virtual machine is running to the virtual machine; providing a first virtual network routing table for the first virtual network, wherein the first virtual network routing table includes, for each virtual machine in the first virtual network, only a single respective entry that specifies (i) a virtual network IP address on the first virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) the only single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; migrating a first virtual machine on the first virtual network from the first host machine to the second host machine; before allowing communication on the first virtual network to the first virtual machine after migrating the first virtual machine, updating the only single respective entry in the first virtual network routing table for the first virtual machine such that the host network address, specified in the only single respective entry, is changed from the first host network address to the second host network address while the virtual network IP address on the first virtual network that is assigned to the first virtual machine, specified in the only single respective entry, is maintained as a same virtual network IP address after migrating the first virtual machine on the first virtual network from the first host machine to the second host machine; identifying a message to be routed on the first virtual network, the message received on the first local area network or the second local area network and including a particular host network address and a particular port; verifying that the host machine assigned the particular host network address is running a particular virtual machine assigned the particular port based on a particular entry in the first virtual network routing table including the particular host network address and the particular port; and determining, from the particular entry, a particular virtual network IP address associated with the particular virtual machine on the first virtual network to which to route the message.

13. The product of claim 12, wherein the operations further comprise: providing a second virtual network routing table for a second virtual network, wherein the second virtual network routing table includes, for each virtual machine in the second virtual network, only a single respective entry that specifies: (i) a virtual network IP address on the second virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) only a single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; and operating the first host machine to run the first virtual machine on the first virtual network while running a second virtual machine on the second virtual network.

14. The product of claim 12, wherein the operations further comprise: defining, using the first virtual network routing table, an IP tunnel for carrying IP traffic between the first virtual machine on the first virtual network and a second virtual machine on the first virtual network, including defining a first endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the first virtual machine and defining a second endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the second virtual machine.

15. The product of claim 14, wherein the first endpoint of the IP tunnel is the first virtual machine running on the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the second host machine.

16. The product of claim 14, wherein the first endpoint of the IP tunnel is the first virtual machine running of the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the first host machine.

17. The product of claim 12, wherein the first local area network and the second local area network are connected by a private network.

18. The product of claim 12, wherein the first local area network and the second local area network are connected by the Internet.

19. A system comprising: one or more computers and one or more storage devices storing instructions that are operable, when executed by the one or more computers, to cause the one or more computers to perform operations comprising: operating a plurality of host machines including a first host machine and a second host machine, wherein each host machine is configured to run one or more virtual machines, wherein the first host machine is assigned a first host network address on a first local area network and the second host machine is assigned a second host network address on a different, second local area network, and wherein the one or more virtual machines are configured to communicate over a first virtual network; for each of the one or more virtual

machines, assigning only a single respective port of the host machine on which the virtual machine is running to the virtual machine; providing a first virtual network routing table for the first virtual network, wherein the first virtual network routing table includes, for each virtual machine in the first virtual network, only a single respective entry that specifies (i) a virtual network IP address on the first virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) the only single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; migrating a first virtual machine on the first virtual network from the first host machine to the second host machine; before allowing communication on the first virtual network to the first virtual machine after migrating the first virtual machine, updating the only single respective entry in the first virtual network routing table for the first virtual machine such that the host network address, specified in the only single respective entry, is changed from the first host network address to the second host network address while the virtual network IP address on the first virtual network that is assigned to the first virtual machine, specified in the only single respective entry, is maintained as a same virtual network IP address after migrating the first virtual machine on the first virtual network from the first host machine to the second host machine; identifying a message to be routed on the first virtual network, the message received on the first local area network or the second local area network and including a particular host network address and a particular port; verifying that the host machine assigned the particular host network address is running a particular virtual machine assigned the particular port based on a particular entry in the first virtual network routing table including the particular host network address and the particular port; and determining, from the particular entry, a particular virtual network IP address associated with the particular virtual machine on the first virtual network to which to route the message.

20. The system of claim 19, wherein the operations further comprise: providing a second virtual network routing table for a second virtual network, wherein the second virtual network routing table includes, for each virtual machine in the second virtual network, a single respective entry that specifies: (i) a virtual network IP address on the second virtual network that is assigned to the virtual machine, (ii) a host network address that is assigned to the host machine on which the virtual machine is running, and (iii) a single respective port of the host machine on which the virtual machine is running that is assigned to the virtual machine; and operating the first host machine to run the first virtual machine on the first virtual network while running a second virtual machine on the second virtual network.

21. The system of claim 19, wherein the operations further comprise: defining, using the first virtual network routing table, an IP tunnel for carrying IP traffic between the first virtual machine on the first virtual network and a second virtual machine on the first virtual network, including defining a first endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the first virtual machine and defining a second endpoint of the IP tunnel using the data specified in the entry of the first virtual network routing table for the second virtual machine.

22. The system of claim 21, wherein the first endpoint of the IP tunnel is the first virtual machine running on the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the second host machine.

23. The system of claim 21, wherein the first endpoint of the IP tunnel is the first virtual machine running of the first host machine and the second endpoint of the IP tunnel is the second virtual machine running on the first host machine.

24. The system of claim 19, wherein the first local area network and the second local area network are connected by a private network.

25. The system of claim 19, wherein the first local area network and the second local area network are connected by the Internet.